

Esercizio 1 - Controllo di una sessione di brainstorming

Indice

Indice.....	1
Introduzione.....	2
Soluzione A.....	2
Metodologia.....	2
Analisi dei costi.....	3
Soluzione B.....	3
Metodologia.....	3
Analisi dei costi.....	4
Formato di input e output con esempi.....	4

Introduzione

Si hanno 2 partecipanti che devono proporre N nomi ciascuno in maniera alternata tale che:

1. la prima lettera del nome coincida con l'ultima lettera del nome proposto immediatamente prima;
2. il nome non sia mai stato proposto prima durante la sessione.

La sessione finisce quando i partecipanti hanno proposto N nomi, dopo la quale il programma deve verificare se la sessione rispetta le regole oppure individuare il primo utente che le viola.

In entrambe le soluzioni ad ogni inserimento dei due utenti viene effettuato il controllo sulle regole della sessione tramite la funzione **canInsert**, successivamente viene memorizzata la stringa sulla struttura dati utilizzata tramite la funzione **insert**, **entrambe le funzioni hanno implementazioni diverse a seconda della metodologia utilizzata**. Alla fine di tutti gli inserimenti il programma comunica se essi sono andati a buon fine, oppure se uno o più utenti hanno violato le regole di sessione.

Soluzione A

Metodologia

Viene utilizzato un **array bidimensionale** come struttura dati per memorizzare le stringhe in **ordine di inserimento**. In questo modo la gestione delle stringhe è molto semplice, come la complessità del codice, allo stesso modo è semplice controllare l'ultima stringa inserita, a discapito dell'efficienza sul controllo dei duplicati:

<i>Vantaggi</i>	<i>Svantaggi</i>
<i>Complessità del codice molto ridotta</i>	<i>Non si utilizzano strutture dati ottimizzate</i>
<i>Controllo ultima stringa inserita costante</i>	<i>Controllo duplicati molto inefficiente</i>
<i>Utilizzo di memoria ridotto</i>	

Analisi dei costi

Analizzando la **complessità computazionale** in particolare si ha costo $O(N)$ per l'**inserimento, annidato al controllo dei duplicati** tramite la funzione `canInsert` che ha costo $O(N)$ di conseguenza il costo **computazionale complessivo** è.

$$O(N) \cdot O(N) = O(N^2)$$

Analizzando invece la **complessità spaziale** si ha un costo $O(N) \cdot O(L)$ dovuto alla **dimensione dell'array bidimensionale**, dove L è la lunghezza massima delle stringhe in input:

$$O(N) \cdot O(L) = O(N \cdot L)$$

Complessità computazionale	$O(N^2)$	<i>Dovuto al controllo stringhe duplicate</i>
Complessità spaziale	$O(N \cdot L)$	<i>Dovuto alla dimensione dell'array</i>

Soluzione B

Metodologia

Viene utilizzata una **hash table con chaining** come struttura dati per memorizzare le stringhe e una **funzione di hashing oaat** (Once At A Time Hash) che prende in input la stringa da memorizzare con la sua lunghezza per generare la **posizione del bucket** in cui deve essere inserita, dopodichè si seleziona la lista del bucket e si posiziona la stringa in cima alla **lista concatenata**. In questo caso, a discapito della **complessità del codice e dello spazio in memoria occupato**, si ha una ricerca molto più **efficiente** e una gestione dei duplicati migliore, inoltre viene utilizzata una **variabile** di appoggio per la memorizzazione dell'**ultima stringa inserita**, questo anche per mantenere un **accesso costante** ad essa:

Vantaggi	Svantaggi
<i>Efficienza nella ricerca</i>	<i>Più spazio in memoria occupato</i>
<i>Soluzione scalabile</i>	<i>Complessità del codice maggiore</i>
<i>Riduzione del costo computazionale complessivo</i>	<i>Necessità di gestire allocazioni dinamiche</i>
<i>Controllo ultima stringa inserita costante</i>	

Analisi dei costi

Analizzando la **complessità computazionale** di questa soluzione si può dire che il costo di tutte le varie funzioni utilizzate, comprese insert e canInsert, è **costante** a differenza della soluzione precedente e questo fa sì di avere costo complessivo $O(N)$ dovuto solamente all'**inserimento da parte degli utenti**

Analizzando la **complessità spaziale** invece si ha costo $O(N) + O(M)$ dovuto, sia al **numero di stringhe in input**, sia al **numero di bucket generati**, dove:

$$M = 2^{HASH\ BITS}$$

di conseguenza il costo spaziale complessivo è $O(N) + O(M)$.

Complessità computazionale	$O(N)$	<i>Dovuto all'inserimento delle stringhe</i>
Complessità spaziale	$O(N) + O(M)$	<i>Dovuto all'inserimento delle stringhe e dal numero di bucket generati</i>

Formato di input e output con esempi

Il formato di input e output è uguale per entrambe le soluzioni, con $n = 5$ si ha:

Input:

Utente 1 inserisci un nome: ciao
Utente 2 inserisci un nome: ciao
Utente 1 inserisci un nome: ontra
Utente 2 inserisci un nome: antro
Utente 1 inserisci un nome: oratorio
Utente 2 inserisci un nome: ora
Utente 1 inserisci un nome: anni
Utente 2 inserisci un nome: inno
Utente 1 inserisci un nome: orca
Utente 2 inserisci un nome: arco

Output:

L'utente 2 ha inserito un nome non valido, chiusura programma...

Input:

Utente 1 inserisci un nome: ciao
Utente 2 inserisci un nome: onore
Utente 1 inserisci un nome: erba
Utente 2 inserisci un nome: amare
Utente 1 inserisci un nome: elefante
Utente 2 inserisci un nome: eleonora
Utente 1 inserisci un nome: arco
Utente 2 inserisci un nome: orca
Utente 1 inserisci un nome: anni
Utente 2 inserisci un nome: inno

Output:

Tutti i nomi inseriti sono validi, chiusura programma...

Input:

Utente 1 inserisci un nome: ciao
Utente 2 inserisci un nome: ciao
Utente 1 inserisci un nome: ciao
Utente 2 inserisci un nome: ore
Utente 1 inserisci un nome: elefante
Utente 2 inserisci un nome: eleonora
Utente 1 inserisci un nome: arco
Utente 2 inserisci un nome: orca
Utente 1 inserisci un nome: anni
Utente 2 inserisci un nome: inno

Output:

Entrambi gli utenti hanno inserito un nome non valido, chiusura programma...